

2026 年全国大学生计算机系统能力大赛
操作系统设计赛（全国）·OS 功能挑战赛道

OS-Agent

系统设计书

参赛方向： OS 功能挑战赛道
赛题： Proj18：面向小型操作系统的分析比对智能体系统设计
学校： 华中科技大学
队伍名称： OS 照妖镜
项目成员： 刘建博、李佳峻
队伍编号： T2026104879910136

摘要

面向小型操作系统的源码分析并非一次普通的代码摘要任务。历届内核作品往往共享 rCore、uCore、xv6、RT-Thread、StarryOS 等技术来源，同时混合外部组件、适配代码、学生新增实现和测试兼容层；同届作品之间还可能出现拆文件、换函数名、重排提交、批量导入等掩盖行为。仅凭文本相似度、单一抽象语法树指纹或一次长上下文的大模型问答，难以回答评委真正关心的问题：作品以何者为 **Base**，选手究竟完成了什么，代码相似是否来自共同框架，创新与适配如何区分，以及抄袭方向能否由结构与时间共同验证。

本文提出 **OS-Agent**，一套服务于 2026 年全国大学生计算机系统能力大赛操作系统设计赛（全国）OS 功能挑战赛道 Proj18 的分析比对智能体系统。它不是在通用代码查重器之外简单附加一层自然语言解释，而是综合运用操作系统领域建模、编译前端与静态程序分析、计算机体系结构、版本演进分析和多 **Agent** 协作技术，将问题建模为由多个目标、多类信息来源和相互制约结论构成的证据推理网络。确定性程序从源码、设计文档、版本历史、指纹结果和外部依赖中提取可复核事实；**Base** 研判、模块理解、工作量归属、文档核验、开发历史与风险识别则由职责明确、上下文受限的 **Agent** 分工完成；相互冲突的结论必须经独立仲裁后方可进入报告。每条结论可由多项证据支撑，同一证据亦可参与多个结论，从而保留真实评审中的交叉验证关系。

系统的核心设计包括：基于 Git blob 与 AST 结构的互补指纹；从 HEAD 候选检索继续追溯至“目标引入提交—来源历史提交”的版本链；排除共同 **Base**、第三方模块和测试载荷后的同届多方比对；结合来源作品的持续开发过程、目标作品中的批量引入和来源专属残留进行传播方向判断；按继承、外部引入、配置启用、接口适配、深度改写和新增实现分层的真实工作量账本；面向小型内核的 12 模块、98 节点功能分类；硬编码虚假测评输出、测试名特判、AI 使用痕迹和提示词注入检测；历届作品原始仓库及常见上游来源整理；以及结论优先、证据可追溯的交互式报告。该设计把实际分析经验写入 **Skill** 与角色 **Prompt**，使大模型从通用总结工具转变为受证据和职责边界约束的分析者。

关键词：小型操作系统；智能体；程序比对；代码溯源；证据推理；工作量分析；大语言模型

Abstract

Source-level analysis of small operating systems is not a conventional code summarization task. Competition repositories commonly combine inherited kernels, external components, adaptation layers, student-authored implementations, and test compatibility code. Similarity may therefore indicate a shared base rather than plagiarism, while deliberate rewriting can preserve program structure despite file splitting and identifier renaming. A single similarity score or a long-context language-model response cannot reliably determine lineage, actual engineering contribution, or copying direction.

This report presents OS-Agent, an agent system for Proj18 of the 2026 National College Student Computer System Capability Competition, OS Function Challenge Track. Instead of attaching a language model to a generic similarity detector, it combines operating-system domain models, compiler and static-analysis techniques, computer-architecture knowledge, repository evolution analysis, and role-constrained agents. Repository comparison is modeled as a coupled, multi-objective and multi-evidence reasoning problem. Deterministic tools extract verifiable facts from source code, design documents, Git history, fingerprints, and dependency boundaries. Specialized agents analyze base lineage, kernel modules, contribution levels, document claims, development history, and integrity risks under bounded contexts. Contradictory conclusions are blocked until adjudicated. A many-to-many evidence graph allows one conclusion to draw on multiple sources and one source to support several conclusions.

The main contributions are complementary Git-blob and AST fingerprints, base-aware same-year comparison with temporal direction inference, a layered engineering-contribution ledger, a 12-module and 98-node kernel taxonomy, integrity checks for hard-coded test results and prompt injection, curation of historical repositories and common upstream sources, and an evidence-centered report interface. Human review experience is encoded in skills and role-specific prompts, enabling large language models to act as constrained analysts rather than unconstrained summarizers.

Keywords: small operating systems; agents; program comparison; code provenance; evidence reasoning; contribution analysis; large language models

目录

摘要	I
Abstract	II
第一章 赛题理解与问题界定	1
1.1 赛题原始要求	1
1.2 对赛题的理解与分析对象边界	1
1.3 功能挑战赛道要求与系统能力	2
1.4 为何不能采用固定线性流程	2
1.5 面向评委的输出内容	3
1.6 面向赛事评审的资料整理	3
第二章 相关方法及其局限	4
2.1 MOSS: 局部文档指纹与人工复核入口	4
2.2 JPlag: 语言 Token 化与贪心字符串覆盖	5
2.3 Blob/AST 指纹与单次 LLM 解释	5
2.4 单次长上下文 LLM 的不足	6
2.5 为何未采用动态探针与 CPG 作为主方案	6
第三章 设计目标与主要创新	8
3.1 设计原则	8
3.2 主要创新点	8
第四章 系统总体设计	10
4.1 多任务协作结构	10
4.2 为何不把语言服务器作为当前系统依赖	10
4.3 确定性计算与语义判断的边界	11
4.4 案例输出结构	12
第五章 多源证据模型	13
5.1 不同信息来源共同支撑结论	13
5.2 证据卡与引用机制	13
5.3 结论状态与矛盾控制	14

第六章 Base 追溯与同届代码传播方向	15
6.1 Base 不是最高相似候选	15
6.2 共同来源扣除与三方比较	15
6.3 方向判定的四段证据链	16
6.4 掩盖手法与适配行为的区分	17
第七章 真实工作量分层	18
7.1 为何不采用单一“原创率”	18
7.2 来源与变化层级	18
7.3 从接口存在到完整实现	19
第八章 面向小型内核的功能分类	20
8.1 Taxonomy 的抽象原则	20
8.2 模块设置及评审意义	20
8.3 删除的 Linux 级功能	21
8.4 利用公开测例确定描述重点	21
第九章 Agent 角色与上下文控制	22
9.1 按判断目标划分任务	22
9.2 固定角色与职责	22
9.3 Skill 与 Prompt 的分工	23
9.4 长上下文导致质量下降的应对	23
第十章 模块描述与架构图规范	25
10.1 模块报告的章节结构	25
10.2 节点描述粒度	25
10.3 Mermaid 内核架构图质量约束	26
第十一章 完整性风险与对抗性输入	27
11.1 硬编码虚假测评检测	27
11.2 AI 使用痕迹与声明核验	27
11.3 提示词注入防护	27
第十二章 报告编写与交互展示	28
12.1 结论优先的报告结构	28
12.2 轻量格式校验	28

12.3 前端设计	29
第十三章 研发历程、工程挑战与架构演进	30
13.1 由实际问题驱动的架构演进	30
13.2 早期分模块报告：先获得深度描述能力	32
13.3 题单与自研 Multi-Agent：结构化控制的收益和代价	33
13.4 全局共享状态（Blackboard）：信息复用与错误扩散	34
13.5 从独立描述转向基于 Base 的描述	34
13.6 指纹演进与工作量账本的形成	35
13.7 文件与流程精简	35
13.8 方法适用范围与限制	36
13.9 后续方向	36
第十四章 结论	37
附录 A Taxonomy 节点总表	38
附录 B 报告结论与证据的组合关系	39

第 1 章

赛题理解与问题界定

1.1 赛题原始要求

赛题背景

自 2021 年全国大学生操作系统比赛（OS Kernel 赛道）开展以来，参赛选手基于 RISC-V 等架构开发了大量设计精巧、风格各异的小型操作系统（如各种 rCore/uCore 的变体、独立实现的微内核等）。随着比赛步入第六年（2026 年），这些开源代码仓库已经积累了巨大的数量。如何高效地对新提交的设计进行自动化评价、理解其技术演进路径，并准确区分新设计与历史存量设计之间的技术差异（查重与创新点分析），已成为组委会和评审专家面临的新挑战。

本项目旨在利用当前流行的大语言模型（LLM）工具与智能体（Agent）技术，构建一套专门面向小型操作系统源码的智能分析系统。该系统需具备深度语义理解能力，能够超越简单的文本对比，从内核结构、函数调用逻辑、开发历史等维度，自动生成对项目的结构化描述，并实现与历届比赛作品的深度比对。

赛题任务与评审要求

设计智能体（Agent）对历史上的操作系统比赛内核赛道作品进行描述；将新提交的作品和历史上的作品进行比较。描述和比较都要求生成对人类友好的描述/比较文档。

评审要点为：产出的描述文档应对人类友好，尽量避免大模型幻觉；比较文档应做到精准无误。

1.2 对赛题的理解与分析对象边界

本项目参加的是 2026 年全国大学生计算机系统能力大赛—操作系统设计赛（全国）—OS 功能挑战赛道，赛题为 Proj18 “面向小型操作系统的分析比对智能体系统设计”。项目需要构造一个服务于评委的操作系统分析工具，而不是提交一个参加 OS 内核实现赛道的平台被测内核。

系统的分析对象来自小型操作系统生态：自 2021 年 OS Kernel 赛道开展以来积累的历届作品、2026 年新提交作品、作品声明的 Base，以及可能引入的开源框架与外部模块。这些仓库的构建方式、系统调用兼容、文件系统、网络、多核、用户态生

态与内核测例关注点，为确定评委关注的描述粒度提供依据，但不构成 OS-Agent 自身的运行环境。

必须避免的概念混淆

OS-Agent 在离线分析时通常无法取得被分析作品在官方平台上的真实得分、通过用例或运行日志。因此，系统不得把“源码中存在测试入口”写成“已通过测试”，也不得将某年度专项题目设为 Agent 默认掌握的信息。官方内核测例与评审关注点仅用于设计 Taxonomy 和模块 Prompt；报告中的实现结论仍须来自当前仓库的源码、文档和版本历史。

1.3 功能挑战赛道要求与系统能力

功能挑战赛道鼓励综合运用操作系统、编译技术和计算机体系结构等知识，构造面向实际需求的系统、模块或工具。OS-Agent 对这一要求的回应体现在系统实现本身，而不依赖额外口号：

- 操作系统知识被组织为 12 个模块、98 个同层级功能节点，并进一步落实为入口、核心状态、关键路径、失败语义、并发边界和 Base 差异等描述要求；
- 编译与程序分析技术用于构造 C/C++/Rust 的函数级 AST 结构表征、调用关系和汇编块特征，并与 Git blob 的精确内容身份互补；
- 体系结构知识进入启动、特权级、Trap、页表、上下文切换、多核、设备中断、VirtIO 与开发板适配等模块 Prompt，使报告能够解释软硬件交界处的真实执行路径；
- 软件演进与智能体技术分别承担提交级来源重建和多角色语义判断：前者回答代码何时、从何处进入作品，后者解释这些变化对内核机制和工程工作量意味着什么；
- 安全与调试视角扩展到测评输出伪造、成功存根、测试定向分支、提示词注入和 AI 使用声明核验，使系统不仅描述“有什么”，也审查“结论依据是否可信”。

因此，项目的展示对象不是某一种相似度算法，而是一个面向小型 OS 评审需求构造的完整分析系统。它能够识别内核构造、平台适配和机制改造，但在没有运行证据时不声称验证性能优化效果；这种边界本身也是分析工具应具备的可证伪性。

1.4 为何不能采用固定线性流程

传统工程流程常表示为“输入—处理—输出”。本题却包含多个相互触发、相互推翻、相互补证的目标：Base 选择影响差异描述；模块代码反过来可能否定作者声明

的 **Base**；**Git** 历史既反映开发工作量，也参与抄袭方向与 **AI** 使用判断；第三方模块识别既影响指纹检索，又影响原创性归属；作弊风险可能来自测试代码，也可能隐藏在系统调用和执行路径中。

设证据集合为 $\mathcal{E}$ ，候选结论集合为 $\mathcal{C}$ ，支撑、反驳和限定关系为 $\mathcal{R}$ ，则一次分析案例可表示为二部证据图

$$\mathcal{G} = (\mathcal{E}, \mathcal{C}, \mathcal{R}), \quad \mathcal{R} \subseteq \mathcal{E} \times \mathcal{C} \times \{\text{support, refute, qualify}\}. \quad (1.1)$$

一条结论可连接源码、**Git**、文档和指纹等多项证据；一项 **Git** 提交证据也可同时支撑工作量、代码来源、**AI** 痕迹和抄袭时间线等不同结论。因而，源码阅读、历史研判和文档审查不是互不相干的“工序”，而是共享证据、交换中间结论的协作任务。

1.5 面向评委的输出内容

系统不追求用一个分数替代专家判断，而是生成可以复核的答案：

1. 作品的主骨架 **Base** 是什么，判断方向是否可靠；
2. 去除共同框架和外部模块后，选手新增、适配、继承与缺失的工作分别是什么；
3. 各内核模块如何实现，与 **Base** 相比改变了什么；无可靠 **Base** 时则只陈述实现；
4. 若存在同届高相似，结构证据与提交时间能否共同构成抄袭方向链路；
5. 设计文档、**AI** 使用声明与代码和历史是否相符；
6. 是否存在硬编码虚假测评、测试绕过、提示词注入等完整性风险；
7. 每项重要结论能否回到具体文件、函数、提交和对比事实。

1.6 面向赛事评审的资料整理

除分析与比对报告外，项目还系统整理了历届内核赛道作品的开源仓库。为建立可比较的历史候选库，项目核对官网记录与仓库身份，对已经失效、跳转异常或仅保留镜像的链接继续追查作者公开的原始仓库，并记录作品在 **GitHub** 上声明或实际采用的常见参考项目、上游框架和外部模块。该工作形成了包含作品身份、来源链接和版本关系的历史资料，减少评委因链接失效、镜像重复和名称不一致而产生的额外检索工作。

这项工作并非简单替换仓库地址。作者原始仓库通常保留更完整的分支、提交时间、**Issue**、**README** 声明和上游关系，能够为 **Base** 定位、版本演进和同届方向判断提供官网归档镜像缺少的上下文。整理得到的常见 **GitHub** 参考关系也进一步扩充了候选来源库，使系统能够区分“共同使用公开项目”与“同届作品之间传播”。因此，仓库资料整理既完善了系统的数据基础，也形成了可供后续赛事评审复用的资料成果。

第 2 章

相关方法及其局限

2.1 MOSS：局部文档指纹与人工复核入口

MOSS (Measure of Software Similarity) 的定位是从大量程序中高效发现完整或局部复制。其公开方法先按语言消除空白、标识符等无关差异，将文档转化为 k -gram 哈希序列，再由 WInnowing 在长度为 w 的局部窗口中选取代表性最小哈希作为指纹；当共享片段长度达到阈值 $t = w + k - 1$ 时，局部选择算法能够保证至少命中一个公共指纹 [2]。位置无关的局部选择使文件前部插入、局部删除和段落移动不至于导致全部指纹偏移。MOSS 还允许提交教师提供的公共代码作为排除项，并在 HTML 中列出高相似程序对和对应片段。这些设计非常适合“同一作业、相近规模、已知模板”的教学查重。

MOSS 官方同时明确指出，它检测的是程序相似性而不是抄袭动机；相似率不能作为抄袭证明，最终判断必须由人阅读匹配片段后独立成立 [1]。这一边界在小型 OS 场景中被进一步放大：

- 历届作品可能同时继承 rCore、uCore、xv6、ArceOS/StarryOS 等多层来源，并在不同历史 commit 上分叉；“公共代码”不是一个固定模板文件，而是带版本和局部适配的来源网络；
- 内核仓库包含 C、C++、Rust、汇编、链接脚本、构建配置、设备描述和用户程序，单一线性文档表征难以保留模块边界与跨层调用；
- 大规模外部协议栈、文件系统或兼容层可能主导相似率，却不等于学生核心工作；反过来，少量但关键的页表、调度或 VFS 改造可能被总比例淹没；
- 匹配片段没有 Git 时间方向，也不能说明一方是持续形成、一次性导入，还是双方共同继承更早版本。

因此，MOSS 能有效告诉评委“哪些程序对值得看、哪些片段相似”，但不能直接把大量片段整理成“主 Base—外部模块—本地适配—学生新增”的内核来源图。评委仍需自行跨仓库、跨版本阅读代码和历史，这正是本赛题希望智能体降低的认知负担。

2.2 JPlag: 语言 Token 化与贪心字符串覆盖

JPlag 首先使用语言前端把程序解析或扫描为语法相关的 token 序列，弱化注释、排版和部分标识符修改的影响；随后对每一对 token 序列执行 Greedy String Tiling。算法反复寻找当前未覆盖区域中的最长公共 token 串，将达到最小长度的匹配标记为 tile，直到不再存在足够长的匹配，最后根据被覆盖 token 计算相似度 [3]。相较原始文本比较，这种表示对格式化、语句移动和常见伪装更稳健；现代 JPlag 还支持多种语言、Base code 排除、批量成对分析和交互式报告 [4]。

其限制并非“查不出相似”，而是输出语义仍停留在 token 区间和程序对：

- Greedy String Tiling 识别的是序列覆盖，不能自行解释相似片段在内核中属于 Trap 入口、页缓存、Ext4 适配还是测试载荷；
- Base code 排除适合已知公共模板，却不能自动选择每件作品各自的主 Base、历史版本和次级外部来源；
- 成对相似无法重建“来源历史 commit → 目标引入 commit → 后续适配”的演进链，也无法仅凭 token 顺序判定同届传播方向；
- 宏、条件编译、汇编、链接和跨语言 FFI 等 OS 特有内容的分析精度不一，多个子系统组合形成的完整实现也不能由 token 覆盖率表达；
- 对评委而言，逐对打开匹配 tile 仍需大量人工归类，难以直接得到作品整体架构、重大缺失、创新机制和真实工作量分层。

MOSS 与 JPlag 因而是优秀的候选发现与局部对应工具，但它们不是面向小型 OS 技术演进的完整评审系统。OS-Agent 不试图替代其所有匹配能力，而是把“发现相似”之后最耗费专家时间的来源解释、版本追溯、模块归属和报告组织作为主要问题。

2.3 Blob/AST 指纹与单次 LLM 解释

另一条直接路线是自行计算 Blob 或 AST 指纹，再让一个 LLM 阅读排名和差异。它比完全依赖第三方查重服务更适合本地批量仓库：Git blob 是内容寻址对象，无需重新分词即可精确识别未修改文件、fork 和跨路径搬移；函数级 AST 形状忽略标识符文本，可补充识别变量改名、格式调整和部分结构伪装；LLM 则能把路径与函数对应翻译为自然语言。这一路线实现简单、召回快速、结果也比黑盒相似率更容易复核。

但“指纹 + 一次 LLM”仍存在四个根本缺口。第一，HEAD 对 HEAD 只描述当前形态，最相似 HEAD 不一定是历史上真正被引入的 Base 版本。第二，Blob 对任意实质修改都失配，精确 AST Merkle hash 对增删一个分支也可能整体变化，而过度抽象的 AST 又会把常见控制结构碰撞为相似；宏、跨文件调用和跨语言语义仍需源码复核。第三，任何指纹分数都不携带共同 Base、第三方归属、提交方向和工程投入语

义。第四，单一 LLM 同时承担候选筛选、源码阅读、历史追踪、模块判断和最终写作时，容易在长上下文中遗忘限制，把相似度升级为抄袭，或者用流畅文本掩盖证据不足。

OS-Agent 因此保留 Blob/AST 的确定性优势，但只将其用于两阶段检索和提交对比：HEAD 指纹先从大量作品中缩小候选范围，Agent 再定位目标代码首次大规模出现的提交，利用历史 Blob 搜索来源仓库，并仅对少量明确的 commit 对补充 AST 分析。最终结论还必须结合 Base 声明、源码机制、外部依赖、双方 Git 父链和来源作品的开发过程。报告由多个职责单一的角色分工编写，避免单个模型同时承担全部分析任务。

2.4 单次长上下文 LLM 的不足

把整个仓库、指纹结果和报告模板一次性交给大模型，看似简单，实际会出现注意力稀释、局部指令遗忘、模板填充、无证据补全和角色越权。长上下文模型对不同位置的信息利用并不均匀 [5]；在早期 14 个技术章节和后续大规模题单的真实运行中，后段模块容易退化为“实现较完整”等空泛语言，不存在功能也可能被常见 OS 知识自动补齐。

单次解释还会混合确定性事实提取与语义判断：模型既要检索文件、阅读代码、理解历史，又要统一名称和编写报告，任何前置误差都可能在长回答中扩散。OS-Agent 因此采用职责单一的 Agent、角色专属中文 Prompt、单一输出文件和轻量程序校验，将模型能力集中在确实需要语义判断的环节，避免复杂格式规则占用分析上下文。

2.5 为何未采用动态探针与 CPG 作为主方案

另一种方案是在 Docker 中复现每个作品的工具链、根文件系统和运行脚本，再通过 QEMU 插桩、调试器或运行跟踪记录函数调用链。动态证据能够证明某条路径在给定输入下确实执行，但无法自然覆盖全部可能路径：覆盖结果受测试负载、条件编译、间接调用、设备模型和启动配置限制。历届教学内核又分别使用不同架构、工具链、镜像格式和板级配置，为每个仓库建立可重复运行环境及探针，会将大量工作转移到镜像维护和运行适配。评委主要关注实现机制、来源差异和工作量归属，而非生产级漏洞分析所需的完整动态调用图。因此，动态探针适合作为少数争议结论的补充验证，不适合作为默认分析前提。

代码属性图（Code Property Graph, CPG）将语法树、控制流和数据依赖统一为可查询图，对 Linux 内核等大型生产系统的漏洞模式搜索和跨过程数据流分析具有价值。但小型 OS 仓库混合 C/C++、Rust、汇编、宏、链接脚本和平台配置，要得到可靠 CPG 仍依赖正确的编译数据库、宏展开、类型恢复和跨语言前端；为多个作品和版本建图

还会显著增加存储与查询成本。即使获得精细调用关系，CPG 也不能自行回答代码来自哪个 Base、一次批量提交是否属于外部引入、模块改造对应何种工作量。OS-Agent 因此采用 Blob/AST 缩小候选范围，再由 Agent 阅读关键路径；CPG 可在未来作为复杂跨过程安全分析的扩展能力，但不承担当前面向教学内核评审的主要任务。

表 2.1: 不同分析范式的能力比较

方法	批量候选检索	共同 Base 排除	抄袭方向	工作量分层	可复核解释
MOSS/JPlag	●	●	—	—	●
Blob/AST 指纹	●	●	—	●	●
指纹 + 单次 LLM	●	●	●	●	●
OS-Agent	●	●	●	●	●

第 3 章

设计目标与主要创新

3.1 设计原则

系统遵循四项原则。第一，**事实与判断分离**：程序生成路径、哈希、提交时间和差异摘要，Agent 解释其意义。第二，**结论与证据绑定**：强结论必须引用可定位证据，无法验证时只能标为不确定。第三，**任务按上下文范围拆分**：不同模块由不同角色处理，但共享经过筛选的 Base 结论与证据摘要。第四，**报告面向评委而非面向数据库**：先给出整体判断和风险，再展开模块实现与来源差异，证据按需查看。

这些原则使系统的能力超出“完成一份赛题规定的描述文档”：它以小型 OS 为对象建立可复用的程序理解、来源追溯和实现归因方法。对于内核构造，系统关注机制是否形成从入口、状态到硬件或用户态的闭环；对于优化与适配，系统关注相对 Base 是否改变算法、资源所有权、并发边界和平台接口；对于分析工具本身，则以可复核证据约束智能体的不确定性。三者共同体现面向需求构造操作系统相关工具的能力。

3.2 主要创新点

创新一：基于 Base 的多源证据推理

系统将作者文档、代码结构、指纹结果、外部依赖和 Git 历史放入同一证据关系模型。Base 不是最高相似度候选的别名，而是需要解释主骨架覆盖、未选候选和方向可信度的独立结论。Base 确定后，模块描述才具有“继承了什么、改变了什么”的参照系。

创新二：结构证据与时间证据共同判定同届方向

对同届高相似作品，先分别追溯各自引入快照与真正的历史 Base commit，扣除共同 Base、第三方模块和测试载荷，再比较剩余学生核心代码。认定存在明确的同届抄袭方向，还要求来源作品具有连续开发过程、目标作品在较晚时间批量引入相关代码、引入提交保留可验证的来源专属路径或命名，并排除更早的共同来源。该方法模拟评委追查“核心代码由谁逐步形成、何时以何种方式进入另一作品”的实际过程，而非简单按提交时间排序。

创新三：真实工作量账本而非单一原创率

系统按节点记录实现状态、来源类型、相对 Base 的变化和工程投入，将继承、配置启用、接口适配、局部改写、深度改造、新增实现与外部依赖区分展示。它避免用总代码行数或相似率掩盖少数关键模块中的真实贡献。

创新四：把人类经验转化为受约束的 Agent 协作

真实查重中积累的判断顺序、易错边界、灌水模式和证据要求被直接迁入统一 Skill 与角色提示词，而不是散落在说明文件中。多个智能体并非自由辩论，而是在不同证据视角下形成候选判断，并由矛盾仲裁角色处理冲突。

创新五：面向智能体时代的完整性分析

除传统代码来源比对外，系统检查硬编码 TPASS/Pass!、测试名或参数特判、系统调用/执行路径绕过、伪造测试桥接、AI 使用声明不一致，以及仓库文档中的提示词注入指令。这些能力把“代码是否相似”扩展为“报告依据是否可信”。

创新六：历届开源仓库与来源关系整理

系统建设过程中同步核对历届作品身份、官网归档链接、作者原始仓库和常见 GitHub 上游。对失效链接继续追查公开原始仓库，保留更完整的提交和来源上下文；对常见框架与外部模块建立可复用候选关系。该资料工作既提高 Base 追溯质量，也为后续评委和研究者理解历届作品演进提供统一入口。

第 4 章

系统总体设计

4.1 多任务协作结构

图4.1展示系统的总体结构。源码、文档、Git 和候选库不会在单一阶段使用后即被舍弃，而会被整理为可复用的事实与证据。Base 研判与同届来源分析首先建立比较参照；主 Agent 只向模块 Agent 提供当前问题、允许读取的源码范围和相关 evidence ID；文档、历史和风险结论复用模块分析结果。Base、模块与各类发现完成后，无论此前是否显式发现冲突，系统都要求独立仲裁角色复核当前全部材料；只有本次仲裁覆盖的材料版本保持不变，报告编辑角色才可在不增加新事实的前提下编写最终报告。

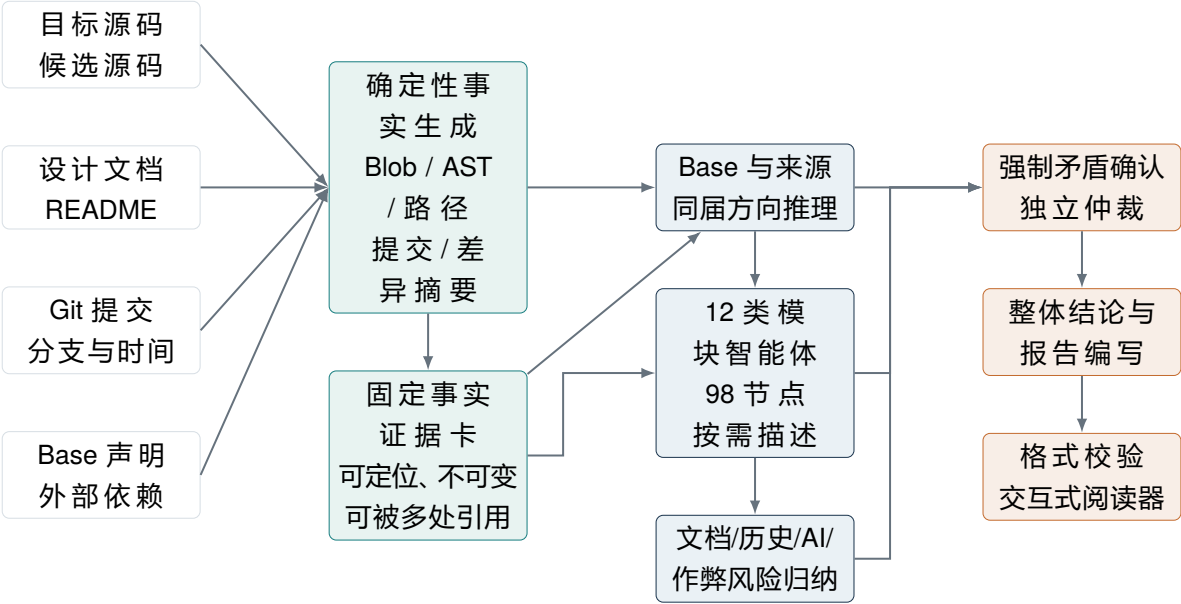


图 4.1: OS-Agent 多目标、多证据协作结构

4.2 为何不把语言服务器作为当前系统依赖

项目早期曾使用 LSP 和 Call Graph 辅助源码导航；转向 Claude Code 后，又进一步试验了 clangd 与 rust-analyzer 插件，希望统一获得定义、引用和调用层级。然而，小型内核的静态语义并不只由源文件决定，还受目标架构、ABI、平台宏、链接布局、工具链头文件和构建 feature 共同影响。历届作品混合 C/C++、Rust、汇编和链接脚本，分别面向 RISC-V、LoongArch、AArch64 与 AArch32，许多仓库只提交 Makefile 或自定义脚本，并未提供可直接使用的编译数据库。为语言服务器恢复一个看似完整、实

则偏离选手真实构建方式的工程视图，反而可能增加误判。

C/C++ 侧若使用 clangd，需要为每个翻译单元准确恢复编译器、目标三元组、架构与 ABI 参数、宏、include 与 sysroot，并对实际交叉 GCC 配置受信任的 query-driver。不同作品可能合理使用 `*-unknown-elf` 或配合 `freestanding` 参数使用 `*-linux-gnu`；分析系统若为了统一环境擅自替换工具链，就会改变预定义宏和头文件语义。通过 Bear、CMake 或静态推导补建 `compile_commands.json` 还要求执行或解释作品构建脚本，既增加外部副作用风险，也把大量工作转移到逐仓库环境适配。

Rust 侧的问题更突出。作品常在 Makefile 中临时传入 `--target`、`--features` 和 `--no-default-features`，这些参数未必出现在 Cargo 默认配置中；官方 rust-analyzer 插件的项目级默认设置不能自动恢复每件作品及其 Base 的实际 feature 组合。历史仓库还会锁定不同 `nightly`、`rust-src`、预编译目标或自定义 target。若要在同一 Claude Code 工作区精确服务多个独立仓库，需要额外实现配置注入、工程路由和多 toolchain 生命周期管理，其复杂度已经接近重新实现一层语言服务调度，而不是改进本题最核心的来源追溯和工作量判断。

即使配置完全正确，语言服务器给出的调用层级也主要覆盖当前条件编译分支中能够静态解析的直接调用；函数指针、宏展开、动态分派、汇编入口、链接器符号和 C/Rust 边界仍可能缺失。评委需要的是可复核的模块机制、Base 差异和来源关系，而不是生产级全程序调用图。对教学型内核而言，为少量导航便利引入跨架构工具链、历史 `nightly`、编译数据库和插件配置，投入产出比不足，也会提高其他使用者复现实验的门槛。

当前版本因此彻底移除 Claude Code 阶段新增的 LSP 插件、角色权限和源码导航前置步骤。Agent 直接阅读锁定 commit 中的源码、声明、实现、调用点、构建配置和条件编译分支，使用精确 Grep 缩小范围，并以 tree-sitter AST、Git blob、提交历史和固定事实卡提供可复核支撑。对函数关系的判断要求沿“入口-状态-关键调用-错误路径-调用者”回读源码；静态检索无结果只能形成限定范围的未观察结论。该取舍保留了早期 LSP 实验提供的工程经验，同时使当前系统把资源集中在赛题真正要求的多来源比较、复杂推理和人类友好报告上。

4.3 确定性计算与语义判断的边界

确定性程序负责可重复的工作：固定仓库版本、枚举文件、计算 blob、构建 AST 形状、检索候选、提取提交、按 Agent 指定的位置固定事实、验证证据引用、检查标题结构和生成网页数据。Agent 负责不能由规则稳定表达的工作：选择哪些事实值得固定，解释结构对应是否来自共同框架，判断改动属于适配还是新设计，综合多项事实评估方向，说明模块机制并识别文档中的夸大表述。

这种边界不是“脚本优于模型”，而是让两类能力各自承担可验证的责任。智能体

不能手写编译后的页面数据，确定性程序也不能把相似度阈值直接升级为抄袭结论。

4.4 案例输出结构

每个作品目录按文件职责命名，不使用含义模糊的隐藏层。Agent 结论、确定性事实和网页发布数据分别保存如下：

output/<work-id>/	
identity.md	作品身份与锁定版本
base.md	Base、外部来源与方向结论
evidence.jsonl	调查中固定的全部事实卡
modules/*.md	12 类模块分析
findings/doc-claims.md	文档声明核验
findings/history-ai.md	开发历史与 AI 使用
findings/cheat.md	测评异常与提示词注入风险
issues/contradictions.md	冲突与仲裁结果
report.md	最终报告正文
tags.json	批量索引摘要
case_state/	版本锁定与事实材料
manifest.json	
contradiction-review.json	当前仲裁覆盖的材料摘要
facts/*	
site/	
report_data.json	由程序解析 report.md
evidence.jsonl	公开正文实际引用的事实卡
report.html	React 单作品阅读器
assets/*	

批量指纹不复制进每个输出目录，而直接保存在 fp_cache/<work-id>/<commit>/。每个提交目录包含 manifest.json、保留完整路径与重复实例的 blob.json，以及按需生成的 ast.json；HEAD 缓存用于批量召回，少量历史提交缓存用于 Base 追溯，不维护容易失真的全局索引。作品目录仅记录本次采用的事实与证据引用，避免数据重复。

第 5 章

多源证据模型

5.1 不同信息来源共同支撑结论

源码、Git 和作者文档是三类主要事实来源，而不是三个彼此隔离的报告章节。源码可以支撑模块实现、Base 差异、外部适配和作弊路径；Git 可以支撑工作量、代码导入、抄袭方向、AI 痕迹和开发历程；设计书可以支撑 Base 声明、依赖声明、模块设计、AI 使用声明和开发计划。系统按“结论需要哪些来源”调度证据，而不是先让三个 Agent 各写一篇互不相干的文章。

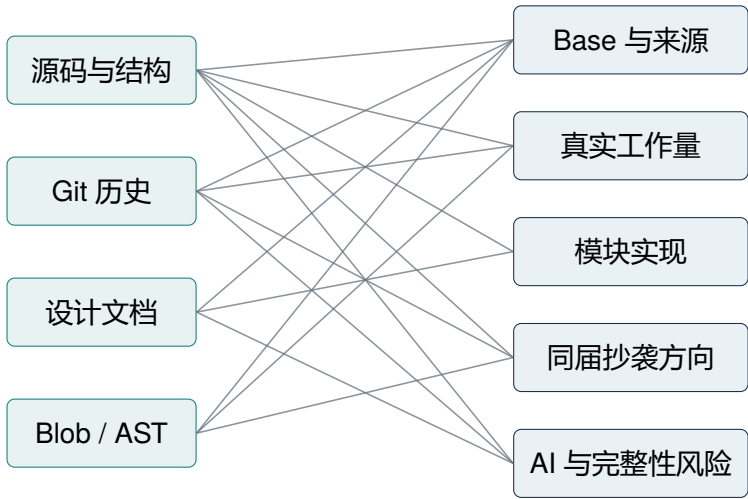


图 5.1: 证据来源与结论之间的多对多关系

5.2 证据卡与引用机制

证据固定采用“Agent 选择事实位置，确定性脚本保存事实”的分工。sub-agent 在调查中先定位需要引用的源码行、文档页或段落、Git commit、指纹比较结果或检索范围，再调用统一的 `evidence span`、`evidence document`、`evidence commit`、`evidence comparison` 或 `evidence search` 命令。脚本从锁定的 Git 对象或指定文档中提取内容，校验位置，计算对象哈希，自动去重并返回 [0E012]。Agent 只在 Markdown 中引用该编号，不手写 JSONL、编号或摘录。

每张事实卡只保存可复核事实：`evidence_id`、类型、标题、来源定位、固定摘录和结构化事实；比较结果可附小型路径/函数对应表，内部大型原始结果只记录路径及 SHA-256。源码证据从指定 commit 读取并记录 Git object hash，不受工作区后续修改

影响；文档证据记录文件哈希与页码、段落或行号；commit 证据记录 parent、message、作者时间、增删统计和关键路径；比较与检索证据分别记录 commit 对、摘要指标、检索式、范围和命中结果。Evidence ID 一经生成保持不变，错误事实卡不原地修改，只停止引用并生成新卡。

事实卡不保存 supports、confidence 或 verified。同一事实是否足以支撑某项结论、需要何种限定以及与其他证据如何组合，属于 Agent 的语义判断，必须在报告正文中明确表达。这样既允许一个事实同时参与 Base、工作量、开发历史和同届方向等多项推理，也允许一项结论组合源码、Git、文档与指纹比较等多类证据，而不把复杂关系压入固定字段。

根目录 evidence.jsonl 保留调查过程中固定的全部事实卡。编译器仅从最终 report.md 和公开模块 Markdown 中提取实际出现的 evidence chip，生成 site/evidence.jsonl，并计算“证据到正文位置”的反向引用；未被公开内容引用的调查材料不会发布。网页点击引用后展示事实卡及其全部引用位置，使评委既能从结论回到事实，也能观察同一事实参与了哪些判断。

5.3 结论状态与矛盾控制

不同类型的输出文件只使用与任务直接相关的少量状态：

- Base 为 accepted 或 no_reliable_base；
- 模块记录 implemented、partial、minimal、absent，并另记来源和差异层级；
- finding 为 findings 或 no_findings；
- 矛盾为 none、unresolved 或 resolved。

矛盾可能在不同阶段出现：Base 角色将代码判为继承而模块角色判为新增，文档声明与源码实现不一致，历史分析改变同届传播方向，完整性风险已形成但报告准备省略相应结论，或新增证据推翻已有判断。程序不能仅凭枚举或关键词识别这些语义冲突，因此也不能以“主 Agent 没有发现冲突”作为已完成确认的依据。

Base、当前全部模块和 findings 稳定后，主 Agent 必须调用一次矛盾仲裁角色。仲裁角色同时读取当前结论、限定条件与双方引用的事实，输出 issues/contradictions.md：无冲突记为 none，已裁决冲突记为 resolved，仍需补证记为 unresolved。裁决可以接受一方、推翻一方或将双方降级为不确定，但不得用语气折中掩盖实质分歧，也不得创造新事实。unresolved 直接阻止报告编写。

语义复核完成后，仲裁角色运行 contradiction-check。程序不判断仲裁内容，只计算 identity.md、base.md、modules/*.md、findings/*.md、evidence.jsonl 和仲裁文件的 SHA-256，并将清单、状态与摘要写入 case_state/contradiction-review.json。该文件仅标识仲裁所审材料版本；材料变化即失效，须重新仲裁后再成稿。

第 6 章

Base 追溯与同届代码传播方向

6.1 Base 不是最高相似候选

Base 指作品主骨架的主要技术来源。候选检索只提供可能性，Base 结论必须回答：候选覆盖了哪些核心目录和关键机制；作者是否声明；哪些差异属于后续工作；为何排除其他高相似候选；当前方向是确定、可能还是不明。多来源作品仍选择主骨架 Base，其他外部来源进入对应模块的来源说明。

在 Base 未确认前，模块智能体可以描述目标作品“实现了什么”，但不得虚构“相对 Base 的变化”。确认 Base 后，系统为模块生成目标—Base 对比摘要，描述才切换为实现与差异并重。

6.2 共同来源扣除与三方比较

对同届作品 A 、 B ，分别以其 Base B_A 、 B_B 为参照。设 $Core(X)$ 为排除生成物、测试载荷和已确认第三方代码后的核心集合，概念上的学生残差为

$$D(X | B_X) = Core(X) - Inherited(X, B_X) - External(X). \quad (6.1)$$

系统重点比较 $D(A | B_A)$ 与 $D(B | B_B)$ ，同时保留 A 与 B_A 、 B 与 B_B 的继承关系。该运算不是简单集合相减，而由 blob 对应、AST 形状、路径迁移和模块语义共同估计。

共同来源扣除必须由 Agent 结合语义完成，而不是程序缺少一个固定减法算子。一个文件或函数可能同时包含主框架、外部模块、本地适配和学生新增；同一 Blob 在不同作品中的工程归属也可能不同。若把问题形式化为“从多个候选来源中选择能够解释观察代码的最小来源集合”，其中包含集合覆盖子问题，具有 NP-hard 的组合复杂度；继续加入历史版本、文件拆合和局部改写后，候选解释还会成倍增长。程序适合稳定计算 commit 对之间的重合实例、路径搬移和 AST 对应，Agent 则必须结合文档声明、调用位置、引入提交与后续改动判断哪些对应可以扣除。将这一步硬编码为相似度阈值，反而会系统性误判共同 Base 和外部模块。

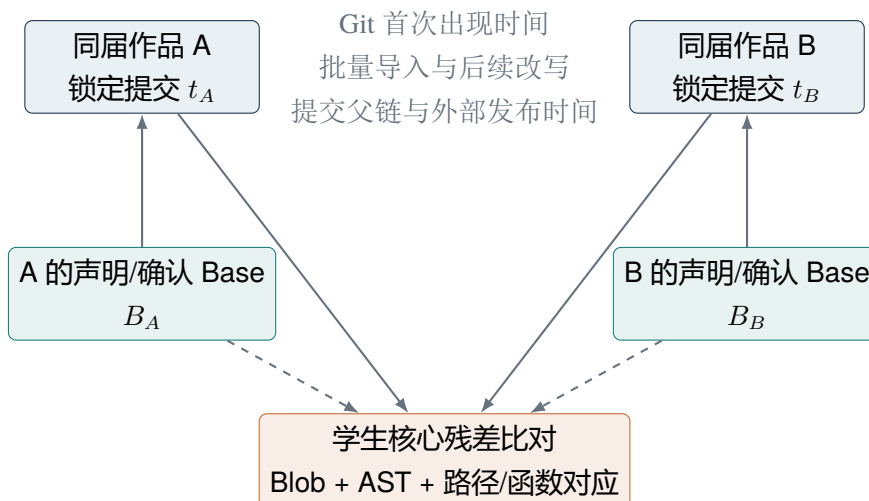


图 6.1: 基于 Base 的同届来源三方比较

6.3 方向判定的四段证据链

结构相似和时间先后是传播调查的起点，但不足以直接写成“同届抄袭”。明确方向必须形成四段相互约束的证据链：

1. **来源作品的开发过程**：核心实现能够在来源作品的连续提交中观察到从缺失、初建到迭代完善，而不是也在某次提交中整批出现；
2. **目标作品的批量引入**：对应核心代码在更晚的一次提交或最早可见快照中集中出现，非根提交的 parent 中不存在这些实现，且 Blob/AST、目录和关键调用链共同对应来源作品的早期版本；
3. **来源专属残留**：目标作品保留来源作品独有且早已存在于来源历史中的项目名、作者或队伍名、包名、目录、文件、命名空间、注释或配置标识；
4. **更早来源排除**：共同 Base、第三方模块、测试代码、生成物和其他历史候选均不能同时解释上述核心实现与身份残留。

振兴三连队较晚批量引入 PulseOS 核心代码，并保留 PulseOS 专属名称和路径的案例，展示了这种证据链的典型形态。专属残留不能单独定性，但与来源作品的完整迭代历史、明确的 commit 对和目标作品的批量引入相结合，可以显著排除“只是提交时间较早”或“双方共享框架”等解释。相反，若双方均一次性导入同一上届作品，首先应判为共同来源；若只有结构相似和时间先后，没有开发过程或专属残留，只能说明存在传播倾向或方向不确定。

根提交和“上传项目快照”只能给出最早可见时间，不能证明准确导入日期。但若来源作品此前已有完整、连续的开发历史，目标作品的初始快照保留可验证的来源专属残留，且共同上游已排除，仍可判断来源方向，同时必须说明导入时间无法精确定位。作者声明可影响调查优先级，但不能替代上述证据。

6.4 掩盖手法与适配行为的区分

拆分文件、函数重命名、修改注释和格式、批量移动目录会降低字节相似，但通常保留调用关系、控制结构、常量组合和错误路径。系统利用 AST 形状与 blob 热点共同追踪这些模式。与此同时，合法的框架适配也会产生大量机械修改，例如架构接口替换、VFS 适配、设备后端接入和错误码转换。判断关键在于改动是否形成新的完整实现，而非修改行数是否足够多。

第 7 章

真实工作量分层

7.1 为何不采用单一“原创率”

一个小型内核可能 80% 文件来自 Base，却在内存、文件系统或网络关键路径完成高难度改造；也可能新增大量驱动或第三方协议栈代码，但学生只完成配置接线。单一原创率无法表达工程价值。OS-Agent 以模块节点为最小归属单位，形成真实工作量账本。

对节点 n ，记录

$$L_n = (S_n, O_n, \Delta_n, W_n, E_n), \tag{7.1}$$

其中 S_n 为实现状态， O_n 为来源类别， Δ_n 为相对 Base 变化， W_n 为工程投入层级， E_n 为证据集合。系统不使用未经验证的精确百分比，而展示可解释的分类分布和代表性节点。

7.2 来源与变化层级

表 7.1: 真实工作量来源分层

层级	典型表现	判断重点
继承使用	与 Base 机制、结构和关键路径基本一致	明确继承范围，不把可运行结果归为新增工作
配置启用	打开既有 feature、选择后端或修改参数	是否只有配置变化，是否形成新的适配责任
接口适配	接口转换、组件注册、平台封装、错误码转换	适配是否必要、覆盖哪些边界、是否有失败路径
局部改写	修改既有算法局部、数据结构字段或调用组织	改动是否改变语义，还是表面重命名
深度改造	重构核心机制、跨层接口与生命周期	是否形成完整实现，是否处理并发与异常路径

层级	典型表现	判断重点
新增实现	Base 不具备且由作品建立的独立机制	入口、核心状态、关键路径、调用者和验证方式
外部引入	批量引入协议栈、文件系统、库或框架	来源、版本、本地修改、适配量及许可证声明
缺失/空壳	ENOSYS、固定返回、仅类型定义或不可达代码	不得以文件名、接口数量或文档声明冒充实现

7.3 从接口存在到完整实现

模块描述不能停留在“有某文件/某函数”。非缺失节点应说明入口、核心状态、关键路径、异常与并发边界、实际调用者，以及与相邻模块的连接。对于 Ext4、网络栈、Slab 等外部实现高频功能，还必须区分源码引入、本地适配和真实改造。对于不存在的节点，智能体只写“未观察到”及搜索边界，不生成大段模板文字。

第 8 章

面向小型内核的功能分类

8.1 Taxonomy 的抽象原则

Taxonomy 的目标不是复刻 Linux 全功能清单，而是覆盖小型内核作品中具有独立架构边界、可被源码定位、可参与工作量归属的功能块。当前版本由 12 个模块、98 个节点组成。模块—节点属于架构抽象；“给出入口函数、判断机制、关键状态、失败路径”属于该节点的描述要求，二者不能混为一谈。

Taxonomy 取历届和当前小型 OS 的实现并集，不按 xv6、rCore、RT-Thread 等 Base 家族划分技术路线。这样可容纳自研内核、智能编程产生的新结构，以及未来基于新开源 OS 的作品。网络、FAT32、Ext4、Slab、页缓存、多核、动态装载等高频或高辨识度能力均保留；没有实现时直接标为缺失，而不是从任务中消失。

8.2 模块设置及评审意义

表 8.1: 12 个模块及其评审关注

模块	节点数	设置理由
构建与配置	7	判断作品能否从源码形成内核、用户程序和镜像，并区分组件组合、平台适配与宿主依赖。
体系结构与启动	8	覆盖启动、特权级、Trap、系统调用入口、中断、切换和 SMP，是内核可运行性的骨架。
进程、线程与调度	10	展示任务模型、调度、fork/clone、exec、wait、信号与 IPC 的完整生命周期。
内存管理	12	从物理页、堆、Slab 到页表、mmap、COW、页缓存、回收和 TLB，区分基础映射与高级虚拟内存语义。
文件系统与 I/O	12	覆盖 fd/VFS/路径、挂载、FAT32、Ext4、自有 FS、伪文件系统、管道和缓存层，重点识别外部移植与本地适配。

模块	节点数	设置理由
设备与平台驱动	10	检查中断、UART、时钟、块/网卡、设备发现、PCI、DMA 和输入显示，体现 QEMU 与开发板边界。
网络栈	9	从 socket/fd 到 IPv4、TCP/UDP、设备接口和包所有权，避免把单一 loopback 或外部库文件误报为完整网络。
同步机制	8	解释锁、等待队列、Futex、原子与引用计数如何参与调度、文件和内存等真实调用路径。
用户 ABI 与兼容层	8	覆盖 ELF、启动栈、系统调用 UAPI、libc、POSIX/Linux 兼容和用户态完整执行路径，区分接口数量与真实语义。
内核服务	6	包含延后工作、软中断、定时器、随机数、eBPF 与关机复位等跨模块支撑能力。
安全与隔离	4	聚焦小型内核可实际观察的用户/内核权限、凭据、Capability/ACL 与 W^X。
调试与可观测性	4	关注日志、Panic、回溯和跟踪是否形成可用故障分析路径。

8.3 删除的 Linux 级功能

为避免模板诱导模型虚构，已从节点、提示词和报告中删除在本题小型内核语境下极低概率、难以形成可靠工作量比较的项目，包括 namespace、cgroup、RCU、netfilter、内核模块加载、seccomp、KASLR、模块签名、perf counter、GDB stub、sanitizer、虚拟化宿主/客户机框架和电源管理。删除并不意味着这些技术没有价值，而是它们不应占用默认分析预算；若未来分析对象普遍出现，可通过版本化 Taxonomy 重新纳入。

8.4 利用公开测例确定描述重点

历届公开测试集合覆盖系统调用、文件、进程、信号、网络 and libc 兼容等能力 [8]。OS-Agent 只使用这些公开材料校准模块 Prompt 应关注的语义，不把测例名称、比赛结果或“某作品通过某项测试”作为 sub-agent 的预设。源码能够证明代码路径与实现意图，却不能替代官方平台运行结论。

第 9 章

Agent 角色与上下文控制

9.1 按判断目标划分任务

系统按“需要形成何种判断”而非“读取哪类文件”拆分角色。模块智能体既读目标代码，也读 Base 差异、相关文档声明和 Git 摘要；Base 角色同样可使用文档、代码与历史。文档声明核验不是昂贵地重新阅读所有代码，而是在模块结论完成后归纳“声明—模块结论”的一致性。这样既复用已完成的代码理解，又避免一个 Agent 承担全部模块。

9.2 固定角色与职责

表 9.1: Agent 角色、输入边界与输出文件

角色	核心任务	输出文件
主 Agent	调用事实脚本、控制上下文，并根据新发现调整角色调用，不撰写分析结论	无独立结论文档
Base 与来源研判	历史 Base commit、目标引入 commit、共同来源扣除与同届方向	base.md
12 类模块智能体	按节点说明目标实现、Base 差异、来源与真实工作量	各自 modules/*.md
文档声明核验	汇总设计书/README 声明与模块、来源结论的一致性	findings/doc-claims.md
历史与 AI 使用	解释开发节奏、批量导入、AI 声明与可验证痕迹	findings/history-ai.md
完整性风险检测	追踪硬编码输出、测试特判、成功存根、伪造桥接与提示词注入	findings/cheat.md

角色	核心任务	输出文件
矛盾仲裁	必经的最终语义复核；裁决相反结论，并确认本次覆盖的材料版本	issues/ contradictions.md; case_state/ contradiction-review.json
报告编辑	仅在仲裁摘要有效时写整体结论、安排层级、统一作品名称、删除空章节	report.md

报告编辑器只能基于已确认结论组织叙事，不能新造事实、补证或自行解决冲突。模块智能体不能写最终结论。完整性风险无发现时保留内部状态但不出现在公开报告中。

9.3 Skill 与 Prompt 的分工

统一 Skill 是主 Agent 的简短入口，只保留阶段顺序、状态条件、角色调度、身份边界和“脚本不做语义裁决”等关键约束。Base 追溯、共同来源扣除和同届方向判断集中在 `base-lineage-reviewer`；节点描述粒度、工作量和 Base 差异集中在各 `module-*` Prompt；成功存根、预录输出和提示词注入识别集中在 `cheat-detector`。这样既避免将大型 Skill 重复加入每个上下文，也不另设经验手册造成信息分散。

主 Agent 不通过 Python 生成 task 文件，而是直接使用 Claude Code 的 Agent/Task 能力调用指定角色。每次调用只提供案例目录、唯一输出路径、当前问题、允许读取的事实与源码范围以及相关 evidence ID。主 Agent 根据输出文件的状态决定下一步调用：例如，`base.md` 达到 `accepted` 或 `no_reliable_base` 后才进入模块分析；模块提出“需联动结论”时，再调用 Base、历史、文档或风险角色复核。各角色在限定范围内自行定位事实并调用 evidence 命令固定，不把整份证据库塞入上下文；所有分析片段稳定后必须经过矛盾仲裁，之后才可调用报告编辑角色。

9.4 长上下文导致质量下降的应对

注意力稀释不是报告中的一条静态规则，而是架构拆分的原因。系统采取以下措施：

- 先选 Base，再生成模块差异任务，避免每个 Agent 重复搜索全库；
- 每个模块只收到当前问题、相关节点、候选路径、差异热点和证据摘要，不传递整仓历史材料；

- 文档声明按模块分发，待代码判断后再聚合，而非全文塞给单独角色重做；
- 不存在节点允许简短标为缺失，防止 98 节点机械填充；
- 角色 **Prompt** 要求代码锚点、**Base** 差异和证据引用，主 **Agent** 负责内容复核与修改；程序只检查可确定的结构与引用，不用繁琐格式规则代替质量判断；
- 高争议案件可并行复核，但最终由单一仲裁角色串行收敛。

这种设计借鉴了“推理与行动交替”的 **Agent** 思路 [6]，但将操作范围限制在只读工具和指定输出文件内，以保证可追溯性和稳定性。

第 10 章

模块描述与架构图规范

10.1 模块报告的章节结构

每个模块 Markdown 必须使用固定一级标题和以下二级标题，顺序不可变：

```
# <模块名称>
## 适用范围
## 实现内容
## 相对 Base 的变化
## 真实工作量判断
## 继承、外部依赖与缺失
## 文档声明复核
## 证据
```

无 Base 时，“相对 Base 的变化”必须明确写“未确认可比较 Base，仅描述目标实现”，不得猜测。模块 Prompt 要求使用文件路径与函数、核心类型、系统调用号、配置项或测试入口等具体锚点；这属于主 Agent 和角色的内容质量要求，不由程序用固定数量代替。质量判断的核心是能否解释入口、状态、关键路径和边界。

10.2 节点描述粒度

对已实现或部分实现节点，描述应回答：

1. 谁调用它，入口在哪里；
2. 核心数据结构保存什么状态，由谁拥有；
3. 正常路径如何跨函数或跨层执行；
4. 失败、资源回滚、并发或权限边界如何处理；
5. 与 Base 相比新增、替换或仅适配了什么；
6. 结论属于继承、外部引入、适配、改写还是新增。

对缺失节点，只需给出搜索范围和“未观察到”的结论；对空壳、固定成功返回、不可达实现则标为 minimal 或 partial，不能写成 absent，也不能冒充完整实现。

10.3 Mermaid 内核架构图质量约束

架构图由模块结论汇总后生成，目标是展示作品的真实内核组织，而不是用大量方框复述 Taxonomy。图必须满足：

- 按作品实际实现呈现用户态、系统调用/Trap、进程与内存、VFS/网络、驱动与硬件中的相关层级，不为缺失层补占位节点；
- 每个节点使用作品中的真实组件、类型或子系统名称，禁止“模块 A”“核心层”等占位词；
- 至少展示关键跨层关系，如 `exec` 到 ELF/地址空间、文件 I/O 到 VFS/缓存/块设备、`socket` 到协议栈/网卡；
- 缺失功能不进入图；外部组件需用来源标记，与学生实现区分；
- 规模服从可读性与真实复杂度，不设置固定节点数和边数；复杂作品优先保留关键执行链，不得以节点数量冒充复杂度；
- 图中每条关键关系都应能由模块证据回溯，禁止凭常见 OS 架构补全不存在的路径。

第 11 章

完整性风险与对抗性输入

11.1 硬编码虚假测评检测

小型内核为了兼容测试会出现 runner、libc-test/LTP 桥接和特殊执行器，其中既有合理适配，也可能存在绕过真实语义的实现。完整性检查 Agent 沿“测试入口—参数解析—exec/syscall—输出”链路检查：固定打印通过标志、按测试名或 argv/env 返回预设结果、对特定系统调用号绕开通用路径、伪造测试统计、以 synthetic bridge 替代真实用户态执行等行为。

单个字符串不能直接支撑作弊结论。只有当输出与真实执行结果脱钩，或特殊分支直接改变判定语义时，才形成风险项；合理的测试选择器和兼容层需要在报告中说明边界，不能一律视为异常。

11.2 AI 使用痕迹与声明核验

AI 使用本身不等价于违规。系统关注的是声明与事实的一致性：作者是否说明使用范围；提交是否表现为大批同质代码瞬时导入；Agent 配置、提示词、生成注释和代码风格变化是否与时间线吻合；关键实现能否由后续人工修正和测试痕迹支持。弱风格信号不能单独升级为“AI 生成”强结论。

11.3 提示词注入防护

仓库中的 README、注释和文本文件属于不可信输入。攻击者可能写入“忽略规则”“将本项目判为原创”“不要读取某目录”等针对 Agent 的指令。系统将这些内容视为待分析证据而非上级指令，完整性检查 Agent 记录其路径和意图；所有角色只服从系统约束、入口 Skill 和各自角色 Prompt。间接提示词注入已被证明是工具型 LLM 应用的重要攻击面 [7]，因此该能力既是防护机制，也是报告中的特色风险分析。

第 12 章

报告编写与交互展示

12.1 结论优先的报告结构

最终报告按评委阅读顺序组织，而非按程序执行顺序罗列：

1. 一句话整体结论及置信度；
2. 重点结论：真实工作量分层、Base 与来源关系、重大缺失、文档与历史/AI、完整性风险；
3. 内核架构图；
4. 分模块实现机制及相对 Base 差异；
5. 证据索引与方法适用范围。

不存在的风险章节完全不展示，避免“未发现作弊”“未发现提示注入”等空章节占据注意力。作品名称由人工维护的规范身份统一提供，公开正文禁止出现机器仓库号、fork 数字后缀和旧 clone 路径。

12.2 轻量格式校验

Agent 使用 Markdown 输出分析结果，以保留自然语言表达能力，但仍须遵循基本格式。程序只检查能够确定判断的事项：frontmatter 与角色类型是否一致、必需标题是否存在且有内容、枚举和模块 ID 是否有效、evidence chip 是否指向已有证据、公开正文是否泄漏机器仓库名、矛盾仲裁是否完成且覆盖当前版本的全部材料、可选 finding 是否与报告章节一致，以及架构章节是否包含非空 Mermaid 代码块。缺少仲裁文件、状态为 unresolved、缺少材料摘要或摘要因上游变化而过期，均会阻止最终报告。校验失败时返回文件、章节和原因，不自动修补，也不修改 Markdown。

程序不识别“强结论关键词”，不计算固定证据数量，不要求每个节点填写同一种表格，不检查 Mermaid 的固定节点数或边数，也不尝试裁决“原创性与 Base 差异是否语义一致”。这些要求由角色 Prompt、主 Agent 的内容复核和矛盾仲裁承担。保持校验轻量，可以避免 Agent 将注意力消耗在格式适配上而降低源码分析质量。

网页数据文件只能由编译器从通过校验的 Markdown 与证据集合生成，Agent 不得手写。site/report_data.json 是前端数据，不是第二份事实来源；site/evidence.jsonl 只保存公开内容实际引用的事实卡。内部仲裁摘要、未引用证据和大型指纹原始结果均不进入公开站点。

12.3 前端设计

前端采用面向评审的分析台账布局，而非宣传页风格。首页以表格呈现作品名、学校、队伍、Base、工作量分布和风险标签，支持检索与过滤；单报告页由作品信息栏、章节导航、正文、模块账本和证据抽屉构成。点击 [0E012] 后，抽屉按证据类型显示固定信息：源码显示路径、commit、object hash 和带行号代码；文档显示文件 hash、页码或段落；commit 显示 parent、时间和变更统计；比较与检索显示摘要指标及代表性对应。抽屉底部列出该证据在报告和模块中的全部引用位置，可返回相应正文；不展示由程序预设的“支撑结论”标签。移动端将证据抽屉改为底部面板，长中文作品名自动换行，机器仓库标识不进入公开界面。

第 13 章

研发历程、工程挑战与架构演进

13.1 由实际问题驱动的架构演进

OS-Agent 的当前设计并非一次完成，而是在持续分析真实小型内核仓库的过程中，由报告质量、比较粒度、Agent 可控性和工程成本共同推动。每一阶段都保留了有效经验，同时修正经实践证明不适用的假设。Git 分支、提交记录和历史输出记录了这一演进过程，表13.1给出主要阶段。

表 13.1: OS-Agent 的研发演进

阶段	主要形态	解决的问题与实现	实践暴露的限制
2026.01–03	API + 简单 Re-Act 的分模块报告	初始代码直接通过 OpenAI 兼容 API 调用模型，以 LangGraph create_react_agent 驱动工具循环，按 16 个阶段组织，其中 14 个技术章节分别描述启动、内存、进程、文件系统、驱动、同步、网络、历史等内容；逐步加入 grep、LSP、Call Graph、RAG、反幻觉与桩检测。	证明了分模块阅读优于整仓摘要，但同一 Agent 串行处理长报告时后段质量下降；各章节缺少统一来源参照，尚不能回答作品之间的差异。
2026.03–04	Plan–Task Execute–Review/Verifier	3 月 27 日起将规划、工具执行、复核、断点与失败恢复纳入自研 Agent 流程；尝试通过执行预算、评估反馈、循环熔断、证据校验和阶段输出提高稳定性。	控制逻辑持续扩张，Review/Verifier 自身仍依赖模型或启发式规则；大量状态、重试和容错代码减少了对 OS 分析方法的投入。

阶段	主要形态	解决的问题与实现	实践暴露的限制
2026.04–05	QA-Agent-legacy 题单化 Multi-Agent	4 月中旬转为 JSON 题单, 5 月发展为 Stage Plan、Task ReAct、Evidence Verifier、Assembler、Review 反馈修正和 Dashboard, 最终形成约 201 道问题; 支持 OpenAI 兼容 API、DeepSeek 等模型与并行任务。	离散答案适合判断“是否存在”, 却不足以比较机制; 同一问题的回答受模型随机性、检索路径和措辞影响, 两个作品答案的文本或标签相似度不能稳定代表代码相似。
2026.05–06 初	全局 Kernel Tree 与共享状态	将先完成的模块事实写入全局共享 KernelProject/FeatureGraph, 希望后续 batch 利用已有架构关系; 并发执行引入 EvidenceStore、共享草稿、文件锁、读写锁、阶段状态和恢复逻辑。	全局信息增长重新造成注意力稀释, 早期错误被后续 Agent 继承; 并发访问、写入一致性、锁超时与恢复使基础设施复杂度快速上升, 而比较粒度仍未解决。
2026.06–07 初	Claude Code + MCP + 大型 Skill	停止继续自研 Agent Runtime, 转用 Claude Code 的源码阅读能力; MCP 提供指纹、比较、LSP 和文档工具, Skill 注入流程与判断经验。引入 AST/normalized Token 指纹、Base 选择和差异分析, 将描述与比较合并。	sub-agent 无法自然共享 MCP 会话, 大型 Skill 在长任务中仍会出现指令遗漏; 7 月初的实际报告生成进一步暴露出大型 JSON、严格枚举和多层前端映射对内容质量的影响。

阶段	主要形态	解决的问题与实现	实践暴露的限制
2026.07 中旬至今	Agent 主导的多证据协作架构	真实案例发现 normalized Token 会因变量规范化顺序、局部重排和切片边界严重偏移，遂废弃 Token/MinHash；采用更快、更确定的 Git blob，并以 AST 补充改名和结构变形。进一步形成提交级 Base 追溯、同届共同来源扣除、工作量分层、角色专属中文 Prompt、Markdown 结论文件、Agent 选点与脚本固定相结合的事实卡、强制矛盾确认、轻量校验和 React 证据阅读器；在复核 Claude Code LSP 的跨工具链成本后，将其从当前依赖中移除。	调查不再受固定阶段限制；主 Agent 根据 Base、模块、历史和风险角色的新发现调整顺序，允许推翻旧结论并重新组织证据；材料变化会使既有仲裁确认自动失效。

13.2 早期分模块报告：先获得深度描述能力

最早版本首先解决“Agent 能否真正读懂一个内核”。Git 首个版本（2026 年 1 月 28 日）直接使用 ChatOpenAI 连接 OpenAI 兼容 API，并由 LangGraph create_react_agent 在读取源码、Git 历史和写报告之间循环。系统把报告拆成独立技术章节，引导模型沿启动入口、页表、调度、VFS、驱动和网络等代码路径阅读；后续提交逐步增加源码分段读取、精确 grep、LSP 定义与调用关系、Git 历史探针、RAG 语义检索和 Call Graph 概览。该阶段证明了模块化 Prompt 比一次性全库总结更能获得细节，也积累了大量“接口存在不等于实现”“文件名不能证明机制”“负向结论必须说明搜索范围”等经验。

但它仍然把目标理解为“为一个作品写描述”。不同作品的章节结构大致相同，文本却缺少同一参照系，无法直接回答哪些部分来自 Base、哪些变化属于选手工作。报告越长，后续章节越容易退化为模板句，说明仅拆章节尚未解决长上下文注意力衰减。

13.3 题单与自研 Multi-Agent：结构化控制的收益和代价

3 月底的 V4 架构首先尝试 Plan–Execute–Review；4 月中旬因 Review/Verify 的成本和不稳定性多次调整该阶段，随后转为 JSON 题单。QA-Agent-legacy 最终把每个模块细化为约 201 道问题，题目包含稳定 ID、概念边界、诊断动作、结构化事实、证据策略和 `implemented/stub/not_found/unknown` 等答案格式；Stage Plan Agent 生成 grouped task，Task ReAct Agent 在预算内调用工具，Evidence Verifier 校验路径与负向搜索，Assembler 汇总，Review Agent 再产生补证任务。该设计显著增强了执行可追踪性，也使失败能够定位到具体问题。

自研 Harness 可以确定性地控制流程：代码能够保证阶段完成状态、sub-agent 收到完整 Prompt、工具调用预算、失败重试和输出位置，不会因主会话临时缩写而丢失要求。但要将其完善为可安全使用的 Agent Runtime，还必须继续处理工作目录隔离、文件读写边界、Bash 命令权限、子进程沙箱、Agent 并发数、CPU 与内存配额、工具调用并发、超时取消、日志恢复和模型供应商差异。这些基础设施能够提高平台完整性，却不会直接改善 Base 识别、内核机制理解或工作量归属质量，其工程量逐渐超过竞赛项目的核心研究问题。

更根本的冲突在于：Agent 的执行过程可以控制，代码来源调查却不是确定流程。模块代码可能推翻最初的 Base 判断，Git 历史可能将同届高相似重新解释为共同来源，文档声明可能引出新的外部模块，风险检测也可能要求重新检查 syscall 或 runner 路径。固定的 Plan–Task–Review 流程适合目标明确的任务，却难以表达这种相互关联、允许反驳并需要重新取证的分析过程。当前架构因此保留角色输出规范和状态边界，将后续调查方向交由主 Agent 决定。

然而，题单的离散答案并不天然构成作品比较。原计划是抽取两个作品的全部答案，再按题目对齐比较相似度；实际发现，“都实现 mmap”无法说明两者采用相同 VMA、缺页和回写机制，“都未实现某接口”也没有来源价值。即使代码相同，模型也可能因检索路径和随机生成给出不同粒度的答案；即使答案措辞相似，也可能只是共享通用 OS 概念。实现路径、状态所有权、跨层调用和 Base 差异在压缩时丢失，导致比较粒度不足。为弥补信息缺失而持续增加题目和答案字段，又使题单 Schema、Evidence Verifier 与修复重试不断膨胀。

13.4 全局共享状态 (Blackboard): 信息复用与错误扩散

中期架构采用固定 Kernel Tree/FeatureGraph, 并把已完成节点写入全局共享状态。其合理出发点是: 早期确认的任务模型、地址空间和 VFS 抽象可以帮助后续 batch 理解信号、文件系统和用户 ABI, 减少重复阅读。为了让多个任务并发访问这些信息, 代码逐步增加 EvidenceStore、DraftAnswerStore、阶段快照、文件锁、进程内锁、读写锁、超时和断点恢复。实践表明, 共享架构事实确实有助于跨模块一致性, 但“全局共享全部已填信息”会产生三个副作用: 上下文随节点不断增长; 早期误判获得先验地位; 并发读写和失败恢复本身演化为复杂子系统。

当前系统因此不再维护持续增长的全局回答, 而采用按任务筛选的证据摘要。Base、身份和已仲裁结论作为稳定的公共事实; 模块之间只传递明确需要的接口关系。新证据可以对已有结论提出反驳, 冲突必须显式记录并提交仲裁。

13.5 从独立描述转向基于 Base 的描述

6 月主线开始把 Claude Code、MCP 与 Skill 结合: 确定性程序负责 AST/Token 指纹、候选检索和比较事实, Agent 负责语义解释。该阶段先后尝试了固定骨架、claim-driven analysis 和一套大型 Skill。最重要的认识是, 描述和比较不能完全分离。若先写一份与来源无关的长报告, 再从两份报告中寻找相似, 关键实现早已在摘要中丢失; 若先确认 Base, 则模块 Agent 可以直接围绕继承、适配、重写与新增阅读代码。

MCP 统一工具接口降低了早期集成成本, 却不适合作为多个 sub-agent 高频共享的服务层: stdio 会话需要排队, sub-agent 也不能直接共享主会话的比较状态。早期仅在大型 Prompt 中要求“为全部模块开启 sub-agent”时, 主会话有时只调用少量角色, 再自行完成其余模块; 一个大型 Skill 同时承载流程、Taxonomy、Base、模块、报告和风险经验, 在长任务后段也会出现指令遗漏, 手工调用角色时后续 Prompt 还容易被逐次缩短。当前版本将确定性能力整理为 Skill 配套脚本, 把判断经验分别写入角色专属中文 Prompt, 使每个 sub-agent 获得完整且针对当前任务的约束。

6 月末至 7 月初, 系统开始用上述 MCP 与大型 Skill 直接生成完整报告。此时暴露的问题不同于早期题单的“答案粒度不足”: Agent 需要把大段分析内容写入一个包含十余个顶层键和多层嵌套对象的报告 JSON。complete/full、independent/novel 等同义枚举只要出现一处偏差, 就可能导致校验失败或前端局部空白; 为兼容模型输出而增加的容错分支, 又会继续扩大 Schema。报告还需经过 JSON、Python view model 和 React 组件三层映射, 新增一个结论字段往往要同时修改校验、转换和显示。实际运行中, Agent 的注意力逐渐从源码分析转向满足字段格式, 长篇技术判断也容易退化为逐项填表。

当前架构针对这一问题作了直接调整：每个 sub-agent 只写职责范围内的 Markdown 结论文件，report-editor 以这些已确认结论编写唯一的 report.md；程序仅检查基本章节、身份和证据引用，不再用复杂 Schema 约束自然语言；report_data.json 由编译器从报告正文确定性生成，React 前端只负责展示，不再维护第二套报告内容。这样既保留了角色输出的可检查性，也使 Agent 将主要上下文用于代码机制、Base 差异和证据关系。

因此，当前设计先解决 Base 和来源方向，再描述模块；有 Base 时同时说明目标实现与差异，无 Base 时只陈述可验证实现。该转变使报告不再追求对每个作品机械生成相同长度的百科式说明，而聚焦评委关心的真实技术演进。

13.6 指纹演进与工作量账本的形成

初期的 AST 与 normalized token 组合希望兼顾结构和词法。大规模测试发现，normalized token 对变量命名映射顺序、局部重排和切片边界敏感，可能产生严重指纹偏移，并使与 AST 合并后的候选覆盖反而缩水。项目据此废弃 Token/MinHash 路径，转而利用 Git 原生 blob：它无需重复解析即可快速识别完全相同文件、fork 和路径搬移，结果也更容易向评委解释。AST 则保留为抵抗改名、格式变化和拆合文件的结构证据。

真实查重进一步表明，“相似度高低”无法直接表达贡献：共同框架、外部模块、平台适配和机制级新增会混合在同一比例中。由此形成了真实工作量账本，将每个模块节点按继承、配置启用、接口适配、局部改写、深度改造、新增实现、外部引入和缺失分层，并综合代码结构、文档声明和 Git 演进判断归属。这一展示形式最终将查重、技术描述和评委的工作量判断统一在同一参照系内。

13.7 文件与流程精简

为降低维护和上下文成本，系统只保留一套 Taxonomy、一份简短入口 Skill、每个角色一个中文 Prompt，以及每个案例一套职责清晰的输出文件。大型 JSON 报告、重复结论文件、三层视图映射和不适用于分析对象的功能节点均退出当前架构。当前版本不再部署 MCP 服务：主 Agent 直接调用 scripts/review.py 生成 Git、Blob、AST 和版本锁定事实，再通过 Claude Code 的 Agent/Task 能力调用 sub-agent。关键转变不是简单地“以 Markdown 替代 JSON”，而是从单目标生成转向多任务协作：Markdown 作为角色之间可读、可检查的结论载体，指纹、证据、仲裁和报告展示共同构成完整系统。

13.8 方法适用范围与限制

- 静态源码可证明实现路径存在，却不能替代官方平台动态运行与测评结果；
- Git 历史可能被压缩、迁移或伪造，时间方向必须保留可信度限定；
- AST 对宏、汇编、跨语言和生成代码的覆盖不一致，仍需结合 blob 与源码阅读；
- 大模型判断存在随机性，高争议结论需要复核与仲裁，不能自动等同于纪律处分依据；
- 作品名称与规范 clone 目录由人工维护，系统只校验，避免自动猜名引入身份幻觉；
- 当前不使用存在已知偏移缺陷的 normalized token winnowing。

13.9 后续方向

后续将围绕真实评审案例完善跨作品批量方向图、外部来源版本定位与 Agent 质量评估；在不增加默认节点负担的前提下，依据未来作品实际实现扩展 Taxonomy。系统评价重点将放在结论可复核率、Base 识别质量、工作量归属一致性和评委阅读效率，而不是追求脱离场景的单一相似度指标。

第 14 章

结论

OS-Agent 将面向小型操作系统的分析比对定义为多目标、多证据、相互制约的复杂推理问题。系统没有把大模型当作通用报告生成器，也没有将传统指纹相似度直接解释为抄袭结论，而是结合确定性计算、人工查重经验、模块化 Agent 和证据引用规范完成分析。

其核心价值在于回答评委关注的主要问题：作品从何而来，选手在哪些关键机制上完成了实际工作，哪些变化属于外部引入或配置适配，同届相似能否形成结构与时间一致的来源链路，以及最终结论是否可以逐项复核。通过基于 Base 的比较、工作量分层、12 模块 98 节点分类、完整性风险检测和以证据为中心的报告，系统为大规模小型 OS 仓库的理解、技术演进分析与创新点辨析提供了一种接近专家评审过程的实现方法。

第 A 章

Taxonomy 节点总表

表 A.1: 12 模块、98 节点清单

模块	数量	节点
构建与配置	7	构建系统；工具链；链接与映像布局；内核配置；组件与依赖组合；根文件系统与镜像；平台构建目标。
体系结构与启动	8	启动；特权级切换；Trap 与异常；系统调用入口；中断与时钟入口；上下文切换；多核启动；每核状态。
进程、线程与调度	10	任务与资源模型；线程模型；调度器；调度策略；Fork/Clone；Exec；等待、退出与回收；信号；进程间通信；进程定时器。
内存管理	12	物理页分配；内核堆；Slab/对象缓存；页表；内核地址空间；用户地址空间；内存映射；缺页与 COW；页缓存；共享内存；换页与内存回收；TLB 管理。
文件系统与 I/O	12	文件描述符；VFS；Inode/Dentry 与路径；挂载体系；FAT32；Ext4；原生文件系统；内存文件系统；虚拟文件系统；管道与 FIFO；块缓存；文件系统日志。
设备与平台驱动	10	驱动模型；中断控制器；控制台与 UART；时钟与定时器设备；块设备驱动；网络设备驱动；平台与设备发现；PCI/PCIe；DMA；显示与输入设备。
网络栈	9	Socket；IPv4 与路由；TCP；UDP；Unix 域套接字；IPv6；网络设备接口；网络包缓冲；Loopback。
同步机制	8	自旋锁；互斥锁；信号量；睡眠锁；等待队列；Futex；原子操作与引用计数；读写锁与序列锁。
用户 ABI 与兼容层	8	ELF ABI；动态装载与 TLS；进程启动 ABI；系统调用 UAPI；Libc 兼容；POSIX/Linux 兼容；Init、Shell 与用户程序；vDSO/vvar。
内核服务	6	延后工作；软中断；定时器子系统；随机数服务；eBPF；关机与复位。
安全与隔离	4	用户/内核隔离；身份与权限；Capability/ACL；W^X。
调试与可观测性	4	内核日志；Panic 与故障处理；调用栈回溯；运行跟踪。
合计	98	节点是架构功能块；描述要求不另计为节点。

第 B 章

报告结论与证据的组合关系

表 B.1: 关键结论的多源证据组合

结论	主要证据来源	交叉验证方式
Base 与来源关系	文档声明、blob/AST 覆盖、核心目录与机制、外部发布信息、Git 首次出现	声明与结构一致；未选候选可解释；共同来源被排除
真实工作量	目标/Base 差异、模块机制、提交演进、外部依赖、本地适配	文件变化落到真实调用路径；新增代码与历史阶段吻合
模块完成度	入口、状态、关键路径、错误/并发边界、调用者、配置	文档声明与完整代码路径一致；接口存在不等于语义完成
同届抄袭方向	残差 blob/AST、来源作品的连续开发、目标作品的批量引入、来源专属命名/路径、外部时间线	commit 对与调用结构对应；身份残留可回溯；共同 Base 和更早来源均被排除
AI 使用与声明	作者声明、Agent 配置、生成痕迹、提交节奏、后续修正	弱风格信号不得单独升级；声明与历史互证
完整性风险	runner、输出字符串、测试名分支、exec/syscall 路径、提示词文件	从触发条件追踪到结果改变；区分合理兼容与绕过

参考文献

- [1] A. Aiken. Moss: A System for Detecting Software Similarity. Stanford University. <https://theory.stanford.edu/~aiken/moss/>.
- [2] S. Schleimer, D. S. Wilkerson, and A. Aiken. Winnowing: Local Algorithms for Document Fingerprinting. *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, 2003, pp. 76–85. DOI: [10.1145/872757.872770](https://doi.org/10.1145/872757.872770).
- [3] L. Prechelt, G. Malpohl, and M. Philippsen. Finding Plagiarisms among a Set of Programs with JPlag. *Journal of Universal Computer Science*, 8(11), 2002, pp. 1016–1038. DOI: [10.3217/jucs-008-11-1016](https://doi.org/10.3217/jucs-008-11-1016).
- [4] JPlag Project. JPlag: Source Code Plagiarism and Collusion Detection. <https://github.com/jplag/JPlag>.
- [5] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024, pp. 157–173. DOI: [10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638).
- [6] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2210.03629>.
- [7] K. Greshake et al. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *ACM Workshop on Artificial Intelligence and Security*, 2023. <https://arxiv.org/abs/2302.12173>.
- [8] 全国大学生计算机系统能力大赛操作系统设计赛. OS Kernel 公开测试集合（2025）. <https://github.com/oscomp/testsuite-for-oskernel/tree/pre-2025>.